

Purpose of this Overview

This document introduces a conceptual systems architecture for technical review and discussion, outlining the context, scope, boundaries, and operational role of AstroGit-OS empowering observatory control, astronomical operations, and engineering teams.

For an interactive breakdown, visit the [web version](#), explore the [AstroGit-OS](#) GitHub repository, or check [real-time development progress](#). Technical feedback and architecture discussions are welcome.

AstroGit-OS — Executive Overview

Declarative Architecture and Operational Lineage for Astronomical Instrumentation via GitOps

Pablo Zúñiga / Industrial & Systems Engineer
Principal Systems Architect / **Cuxhaven Labs**
v0.2 – Systems Architecture Concept
September 28, 2026

EXECUTIVE SUMMARY

AstroGit-OS is a reference declarative architecture for the configuration, maintenance, and operational lineage of optomechanical instrumentation in next-generation astronomical observatories. Applying GitOps and Infrastructure-as-Code principles, AstroGit-OS dispenses with massive astronomical data and focuses exclusively on the operational metadata required to govern scientific infrastructure.

Hardware State Governance in Frontier Environments

Complex physical infrastructure —from ground-based astronomical observatories to orbital payloads and autonomous industrial facilities— operates under extreme conditions where thermal, mechanical, and structural variations directly impact operational integrity. Historically, hardware states have been fragmented across legacy control loops, localized databases, and manual logs, creating blind spots in drift detection, root-cause diagnostics, and states reproducibility.

AstroGit-OS addresses this universal bottleneck through a domain-agnostic declarative architecture. By decoupling hardware operational metadata from high-throughput payload streams, this architecture delivers an immutable framework engineered for the operational realities of next-generation astronomical observatories —such as ELT, GMT, and Rubin— and designed to scale across high-consequence verticals:

- ▶ **Ground-Based Scientific Megainfrastructures**

Optomechanical instrumentation and observatory control networks requiring exact technical reproducibility and non-intrusive drift detection without impacting real-time execution.

- ▶ **Space & Orbital Assets**

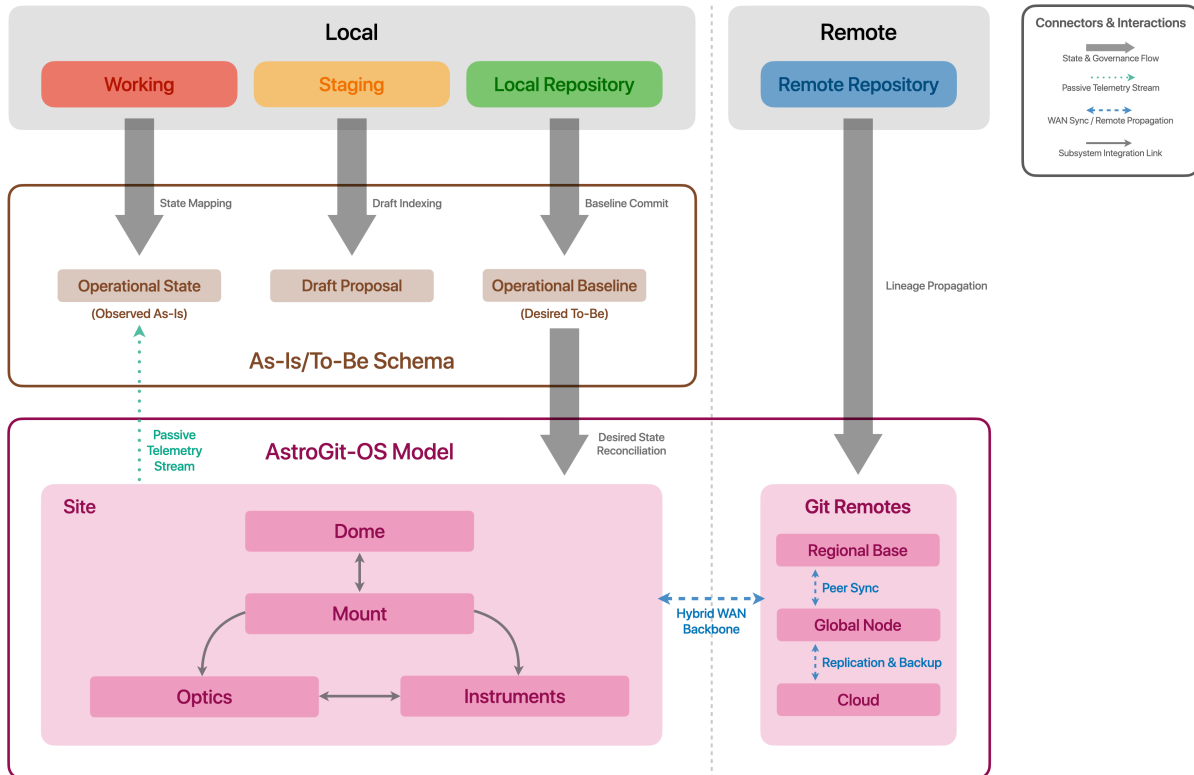
Space-based telescopes, satellite constellations, and autonomous payload environments where off-grid resilience, low-overhead sync, and immutable state lineage are critical.

- ▶ **Aerospace & Industrial Automation**

Automated launch infrastructure, ground support equipment, energy grids, and autonomous mining operations subject to severe network partitioning and strict uptime requirements.

Scope and Boundaries of AstroGit-OS

AstroGit-OS does not capture, store, process, simulate, or transmit astronomical data, nor does it operate telescopes in real time. Its function is to govern a versioned, asynchronous digital representation of the operational state of telescopes and other scientific infrastructure, decoupling scientific activities from engineering operations. **AstroGit-OS records, stores, and audits the evolution of infrastructure configuration through Git as a version-controlled governance mechanism**, keeping the declarative representation of operational state separate from its physical implementation across the instrumentation.



DRAFT FOR TECHNICAL REVIEW ONLY — Cuxhaven Labs

AstroGit-OS Differentiating Principles

- ▶ **Scientific Lineage Without Massive Data**

Associates each astronomical observation block with a SHA-256 identifier of the hardware operational state, enabling verifiable scientific traceability without burdening the repository with massive datasets.

- ▶ **Distributed Operational Resilience**

Supports operational continuity and the recovery of versioned configurations at remote high-altitude observatories through decentralized and synchronized Git topologies.

- ▶ **Non-Intrusive Declarative Governance**

Translates infrastructure maintenance into an auditable workflow through Pull Requests and human validation, operating strictly asynchronously without interfering with real-time telescope operations.

Operational Value in Scientific Infrastructure

Through an operational state reconciliation architecture, AstroGit-OS governs the evolution of infrastructure and instrumentation, detects deviations between observed state and desired configuration, and establishes a verifiable link between observation blocks and the versioned operational state through passive telemetry and non-intrusive state extraction agents. In doing so, it aims to reduce operational uncertainty, minimize downtime, and facilitate the reproduction of technical conditions underlying the scientific use of astronomical instrumentation.

- ▶ **Operational Uptime Optimization**

Minimizes unplanned downtime through early detection of drift between the infrastructure's desired state (To-Be) and its actual telemetry (As-Is).

- ▶ **Technical Reproducibility Assurance**

Facilitates the exact reconstruction of the optomechanical environment by immutably binding each observation block to a SHA-256 cryptographic digest of its operational state.

- ▶ **Standardization of Engineering Interventions**

Streamlines the creation, execution, and verification of engineering procedures by routing hardware modifications through declarative GitOps workflows and peer review.

- ▶ **Immutable Forensic Traceability**

Delivers a continuous, unalterable history of the instrumentation lifecycle via append-only logs, simplifying technical audits and root-cause failure diagnostics.

Declarative Schemas and State Serialization

AstroGit-OS adopts a dual-format strategy for operational state representation to ensure system interoperability, high-throughput machine consumption, and seamless human governance. This architecture separates the human-defined infrastructure specification from the automated collection of live state metadata.

▸ Desired State (To-Be) via YAML

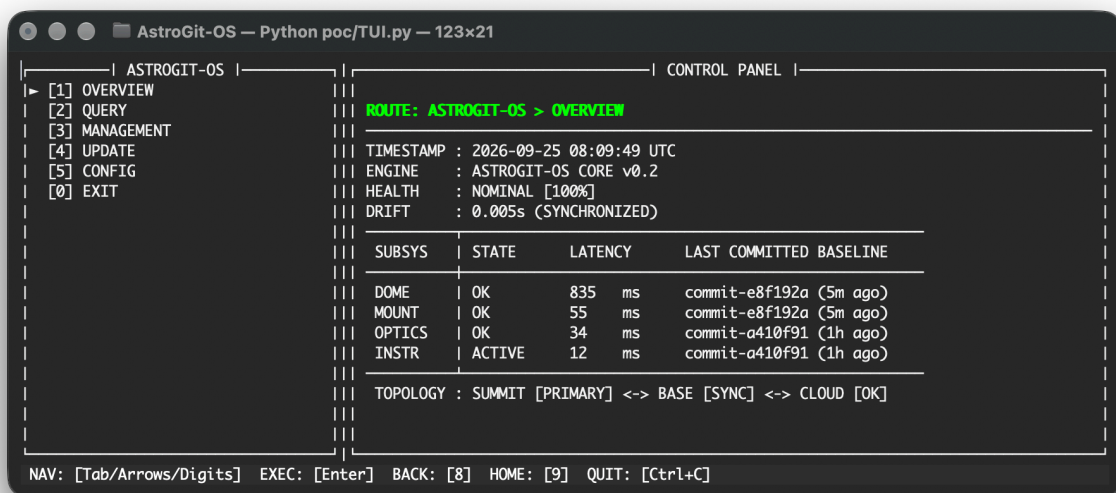
Represents the canonical baseline specifications and composite subsystem configurations. Defined, reviewed, and approved by engineering teams via GitOps workflows, YAML provides optimal human readability, inline documentation, and native compatibility with code review tooling (git diff and Pull Requests).

▸ Observed State (As-Is) via JSON

Captures the live operational state vector extracted directly from infrastructure telemetry. Generated automatically by non-intrusive extraction agents, JSON offers a lightweight, low-overhead format optimized for high-speed machine parsing, API consumption, and automated drift reconciliation.

TUI Console and Operational State Management

AstroGit-OS integrates an interactive TUI —compatible with Git commands and standard command-line utilities— for querying, monitoring, and auditing astronomical infrastructure, combining the precision of its declarative model with command-line agility. From this interface, operators manage the operational state and drift between the baseline configuration recorded in Git (To-Be) and the physical equipment’s actual state (As-Is), streamlining audits, maintenance routines, and state reconciliation across the Summit, Base operations, and Cloud.



```
AstroGit-OS — Python poc/TUI.py — 123x21
┌───┴───┐
│ [1] OVERVIEW |
│ [2] QUERY    |
│ [3] MANAGEMENT |
│ [4] UPDATE   |
│ [5] CONFIG   |
│ [0] EXIT     |
└───┴───┘

||| ROUTE: ASTROGIT-OS > OVERVIEW |||
|||
||| TIMESTAMP : 2026-09-25 08:09:49 UTC
||| ENGINE    : ASTROGIT-OS CORE v0.2
||| HEALTH    : NOMINAL [100%]
||| DRIFT     : 0.005s (SYNCHRONIZED)
||| | | | |
||| SUBSYS | STATE | LATENCY | LAST COMMITTED BASELINE |
|||-----|-----|-----|-----|
||| DOME   | OK    | 835 ms  | commit-e8f192a (5m ago) |
||| MOUNT  | OK    | 55 ms   | commit-e8f192a (5m ago) |
||| OPTICS | OK    | 34 ms   | commit-a410f91 (1h ago) |
||| INSTR  | ACTIVE| 12 ms   | commit-a410f91 (1h ago) |
|||-----|-----|-----|-----|
||| TOPOLOGY : SUMMIT [PRIMARY] <-> BASE [SYNC] <-> CLOUD [OK]
|||
NAV: [Tab/Arrows/Digits] EXEC: [Enter] BACK: [8] HOME: [9] QUIT: [Ctrl+C]
```